

IS 2202 : ADVANCED DATA STRUCTURES AND ALGORITHMS

1. INTRODUCTION TO TREE DS AND BALANCING TREES WITH DSW

Dr. Yohani Ranasinghe
University of Colombo School of Computing



ANSWER

1. What is a Data Structure ?
2. What is an Algorithm ?
3. What are the criteria to select a Data Structure?
4. Why need to learn Data Structures and Algorithms?



DATA STRUCTURE

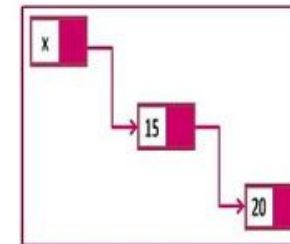
A data structure is a way of **organizing** and **storing** data in a computer so that it can be accessed and modified efficiently.

Why Use Data Structures?

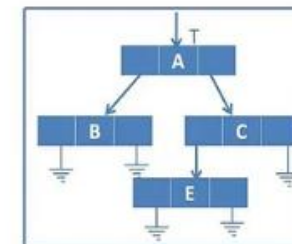
- To store data efficiently
- To access data quickly
- To modify data easily
- To solve problems more effectively (e.g., searching, sorting, scheduling)



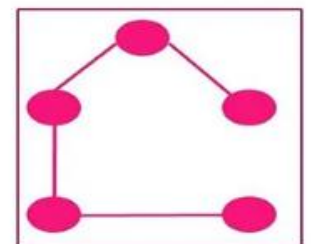
Sorting



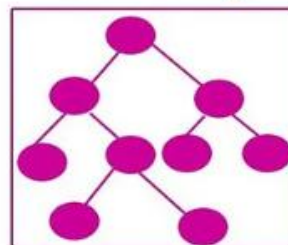
Link list



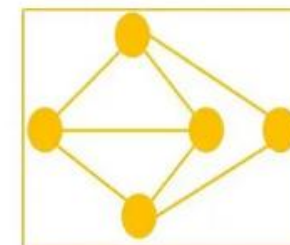
list



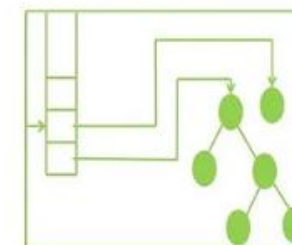
spanning tree



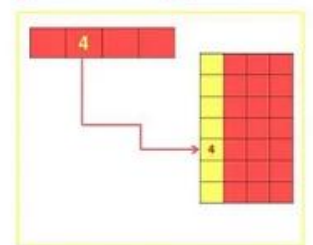
Tree



Graph



Stack



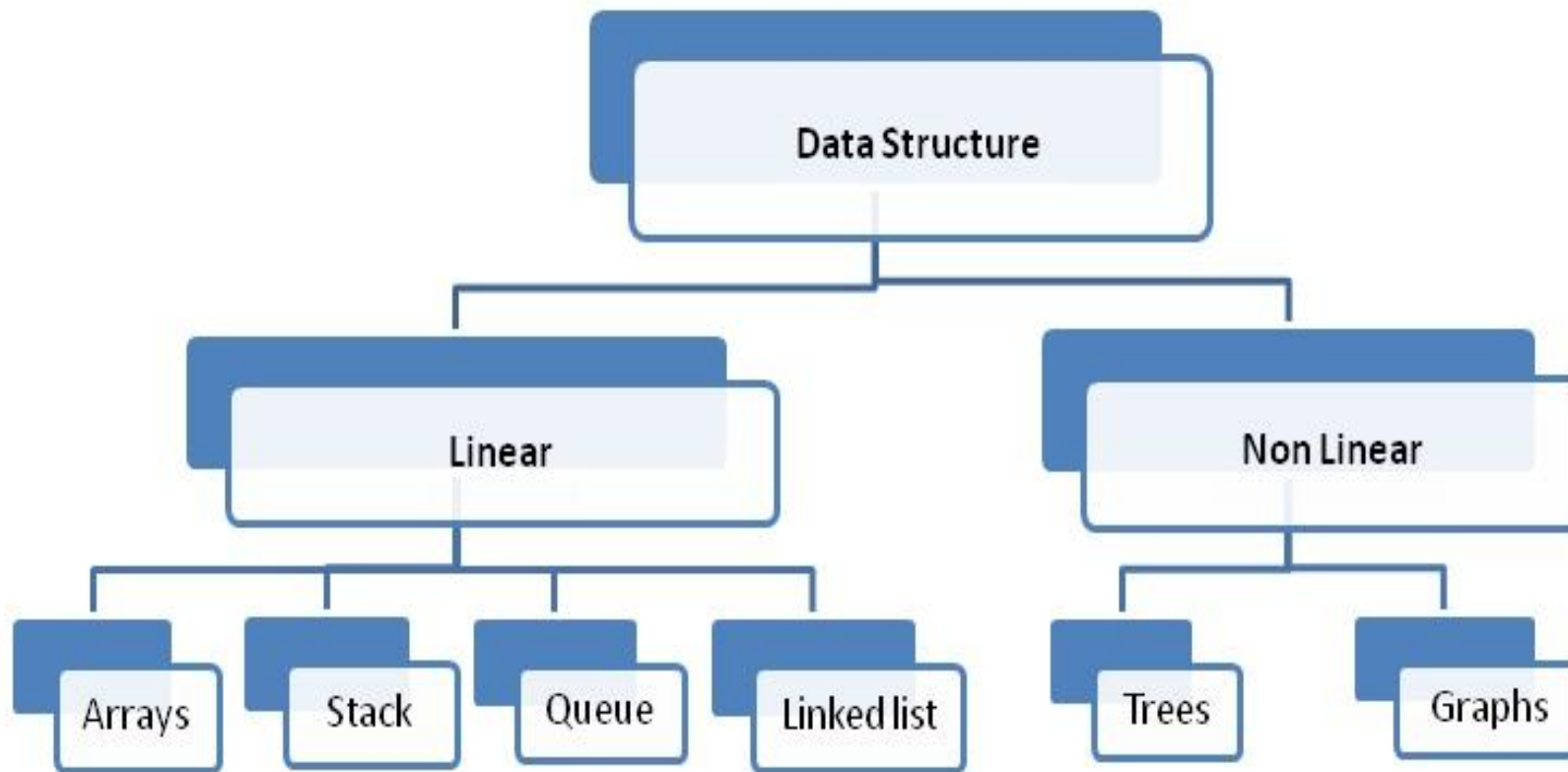
Hashing

ACTIVITY

TAsk	Data Structure
Fast Key Lookup	
Undo Feature	
Print Queue	
Database Index	

There is no single best data structure. The best data structure depends on which operations must be performed efficiently.

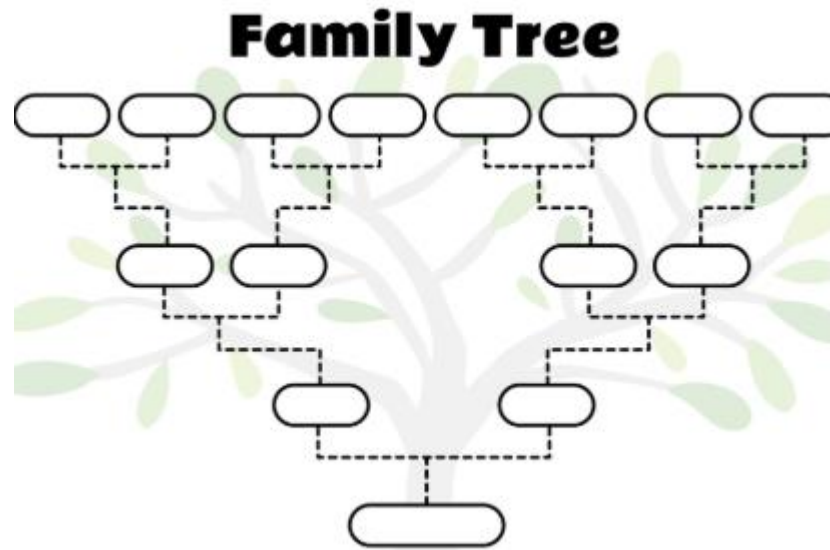
LINEAR AND NON-LINEAR DATA STRUCTURES



Write down details about your family members including relations and ancestors ?

- Mother, Father, Sister, Brother, Grandmother

How to organize it differently?

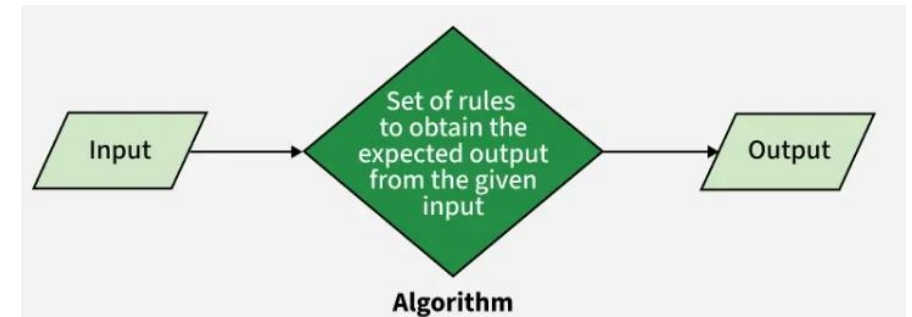


ALGORITHMS

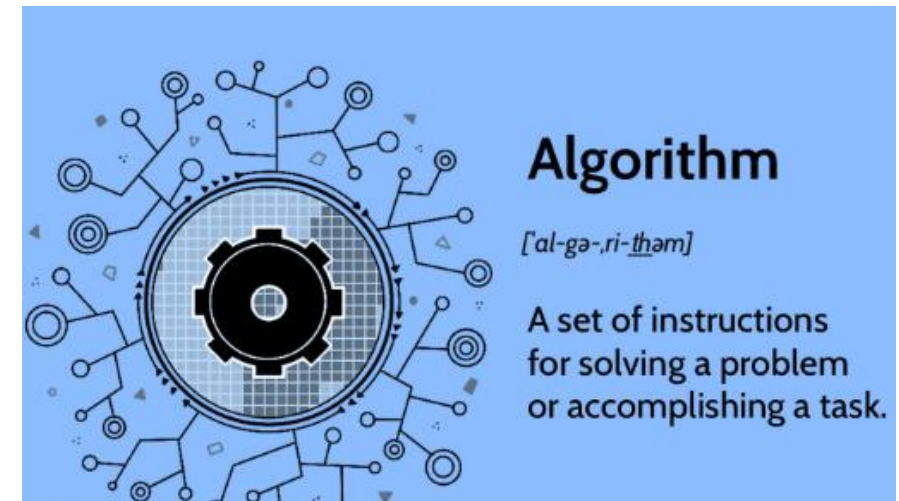
An algorithm is a **step-by-step set of instructions** designed to perform a specific task or **solve a particular problem**.

Why Are Algorithms Important?

- They automate problem-solving.
- Help make software fast, efficient, and reliable.
- Are the foundation of all computing systems—from apps to AI.

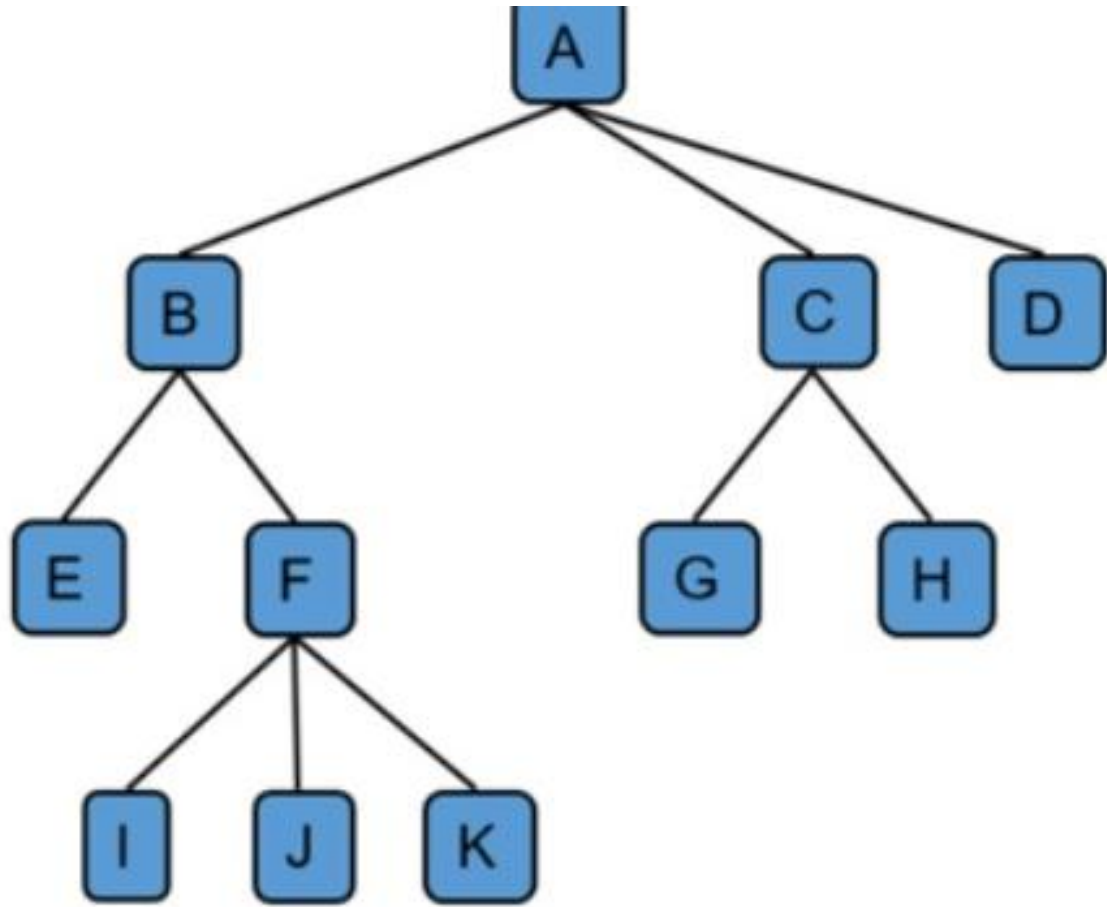


Task	Example Algorithm
Sorting numbers	Bubble Sort, Merge Sort
Searching in a list	Linear Search, Binary Search
Finding shortest path	Dijkstra's Algorithm
Encryption	RSA, AES



Algorithms - Different Aspects

- Correctness of an algorithm
- Different solutions to the same problem
- Complexity
 - Time Complexity
 - Space Complexity



TREE DATA STRUCTURES

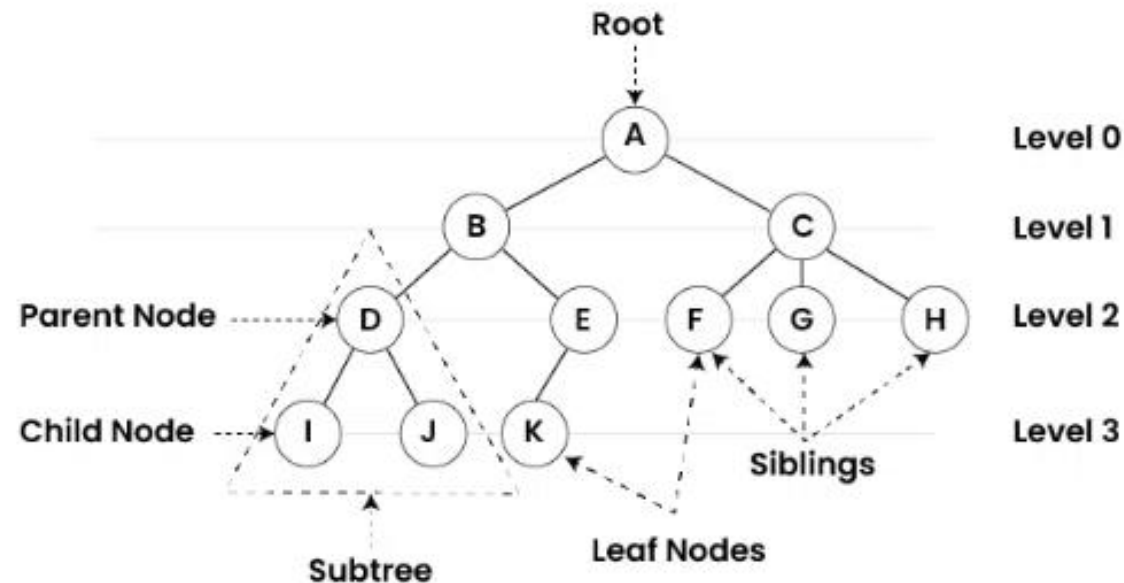
- Tree
- Binary Tree
- Binary Search Tree
- Multiway Tree

TREES

A tree is a **non-linear, abstract** data type with a hierarchy-based structure used to represent and organize data in the form of **parent child relationship**.

It consists of **nodes** (where the data is stored) that are connected via **links**.

Ex: Folder structure
HTML tags



TERMINOLOGY

Path – Path refers to the sequence of nodes along the edges of a tree.

Root – The node at the top of the tree is called root. There is only one root per tree and one path from the root node to any node.

Parent – The node which is an immediate predecessor of a node .

Child – The node which is the immediate successor of a node.

Leaf – The node which does not have any child node is called the leaf node.

Subtree – Subtree represents the descendants of a node.

Levels – The count of edges on the path from the root node to that node. Level of a node represents the generation of a node. If the root node is at level 0, then its next child node is at level 1, its grandchild is at level 2, and so on.

TYPES OF TREES

There are three types of trees –

General Trees

General trees are unordered tree data structures where the root node has minimum 0 or maximum 'n' subtrees.

Binary Trees

In a binary tree, each node can have a maximum of two children linked to it

Are general trees in which the root node can only hold up to maximum 2 subtrees: left subtree and right subtree.

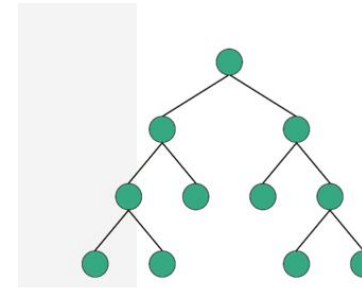
Binary Search Trees

In the BST values of left subtree are always less than the values in the root node and the values in right subtree are always greater than the values in the root node

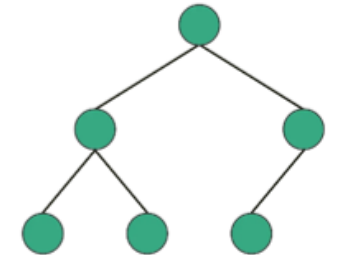
TYPES OF BINARY TREES

Full Binary Tree

A full binary tree is a binary tree type where every node has either 0 or 2 child nodes. Leaves may not all be on the same level. Not all levels need to be fully filled



Full

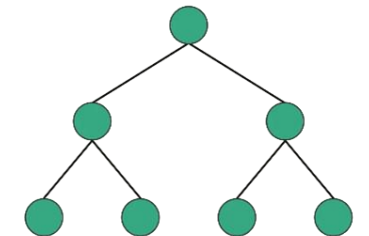


Complete Binary Tree

A binary tree where all levels are completely filled, except possibly the last, and all nodes in the last level are as far left as possible.

A complete binary tree is a binary tree type where all the leaf nodes must be on the same level. However, root and internal nodes in a complete binary tree can either have 0, 1 or 2 child nodes.

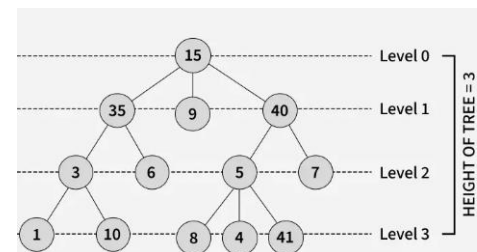
Complete



Perfect Binary Tree

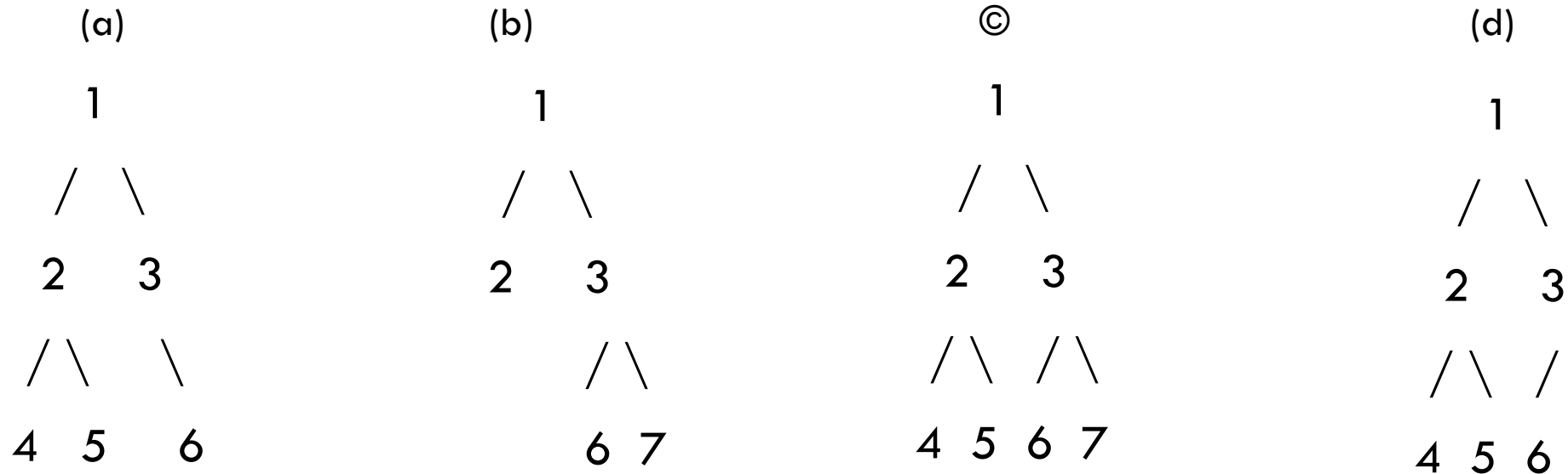
A perfect binary tree is a binary tree type where all the leaf nodes are on the same level and every node except leaf nodes have 2 children.

Number of nodes = $(2^{h+1}) - 1$ (where h is the height of the tree).



Perfect

ACTIVITY



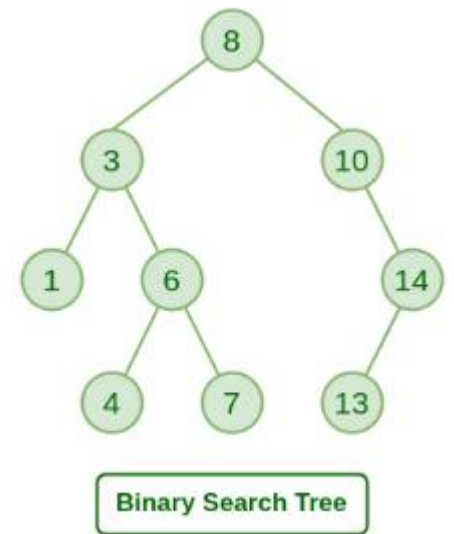
- A. Not Full (node 3 has only one child)
- B. Full - All internal nodes have 0/2 children
- C. Perfect - All leaves at same level, all internal nodes have 2 children
- D. Complete- Filled left to right

BINARY SEARCH TREE (BST)

A BST is a binary tree where each node has at most two children, and for each node, the left child's value is smaller than the node's value, and the right child's value is greater.

An **AVL Tree** is a self-balancing BST where the heights of the two child subtrees of any node differ by no more than one.

A **Red-Black Tree** is a highly efficient self-balancing binary search tree.



BST EXAMPLE

25,37,28,16,7,44,23,31,47,17,26,24,11,1,38

ACTIVITY

You are given the following sequence of integers to insert into an initially empty Binary Search Tree (BST):

[50, 30, 70, 20, 40, 60, 80]

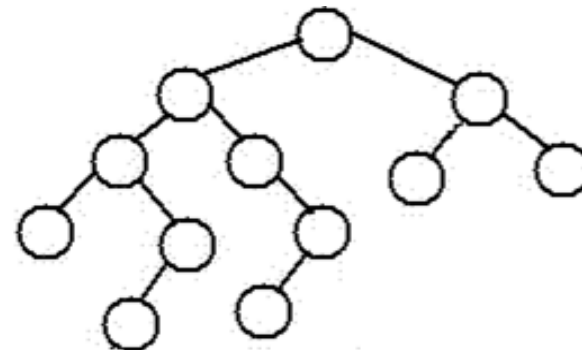
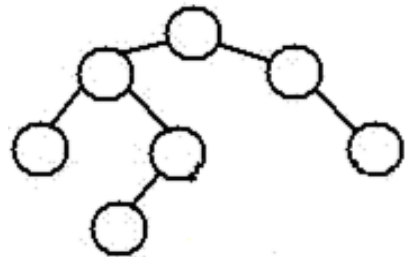
1. Draw the BST after all insertions.
2. What is the result of the following traversals?
 - In-order traversal
 - Pre-order traversal
 - Post-order traversal
3. What would be the height of the BST?
4. If the value 30 is deleted from the BST, what does the tree look like afterwards?
5. C code to construct and traverse this BST (homework)

HEIGHT BALANCED BINARY TREES

The height of a tree is defined as the number of edges on the longest path from the root node to a leaf node.

Alternatively, it is the maximum depth of the tree, where the depth of a node is the distance from the root to that node.

Consider a Binary Search Tree; with 'm' as the height of the left subtree and 'n' as the height of the right subtree. If the value of $m-n$ (balancing factor) is equal to 0, 1 or -1, it is said a balanced binary search tree.

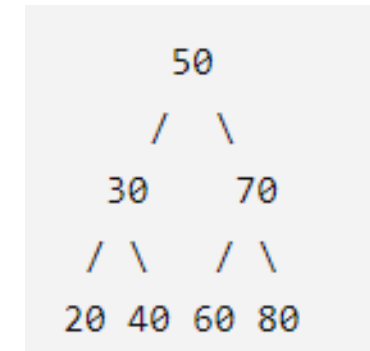


LOCAL VS GLOBAL BALANCING

Balancing in trees refers to the process of ensuring that the height difference between the left and right subtrees of every node in the tree remains minimal, typically to optimize search, insertion, and deletion operations.

1. **Global Balancing**
2. **Local Balancing**

LOCAL BALANCING



Focuses on maintaining balance around individual nodes or small sub-trees.

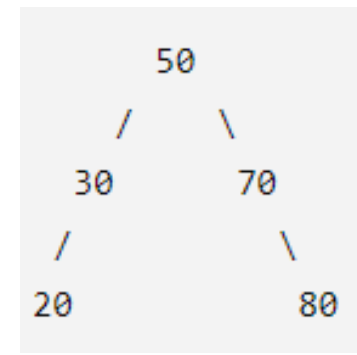
Balancing operations (like rotations) are performed more frequently, typically after each insertion or deletion.

When Used: After an insertion or deletion, if the balance factor at a node violates the tree's balance condition, a local rotation (e.g., left or right rotation) is applied at that node.

Example: In an AVL tree, if inserting a node causes the balance factor of a node to become ± 2 , a single or double rotation is applied at that node only

Goal: Restore the balance locally without examining or altering the entire tree.

GLOBAL BALANCING



Aims to balance the entire tree structure, often achieving an optimal or near-optimal balance.

Complete restructuring of the tree is done periodically, rather than after every insertion or deletion.

When Used: When balance needs to be maintained across multiple levels or subtrees.

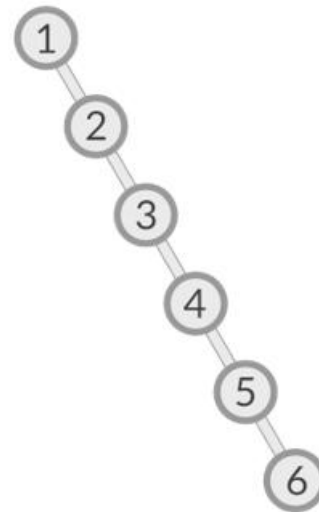
Example: DSW trees

- Rebuilding a binary search tree into a completely balanced tree using an in-order traversal followed by reconstructing the tree from the sorted list.

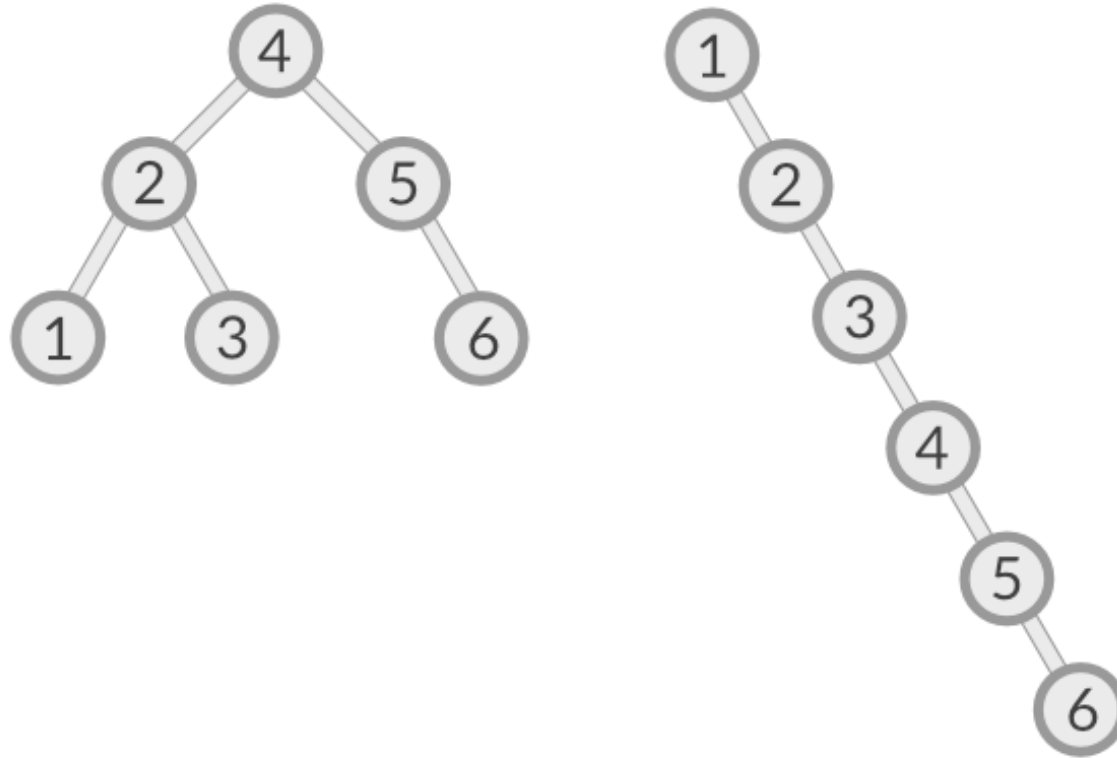
Goal: Achieve a globally balanced tree where all paths from the root to the leaves are roughly equal in length.

DSW algorithm focuses on producing a well-balanced tree **globally**, rather than continuously maintaining balance after every insertion or deletion.

Is this BST?



BALANCED VS. UNBALANCED

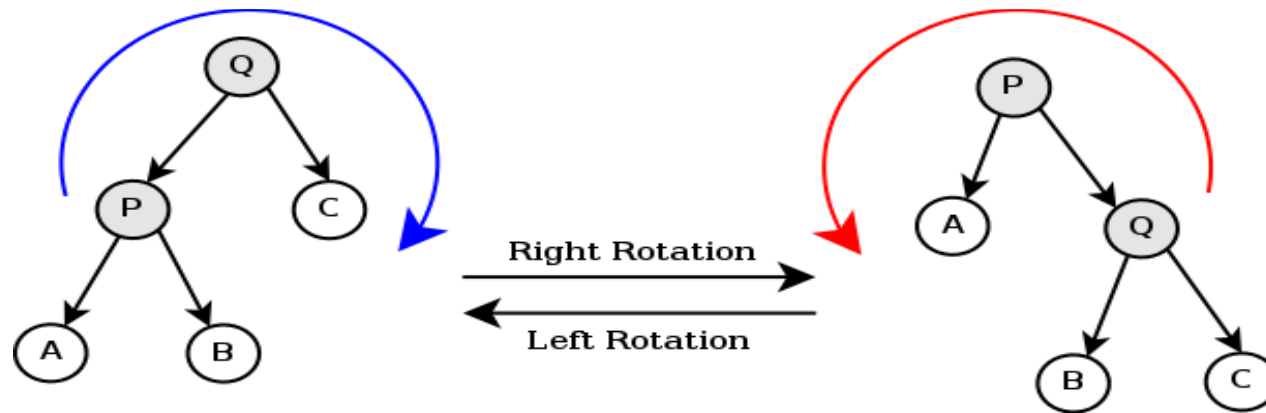


Balancing keeps the tree height small.

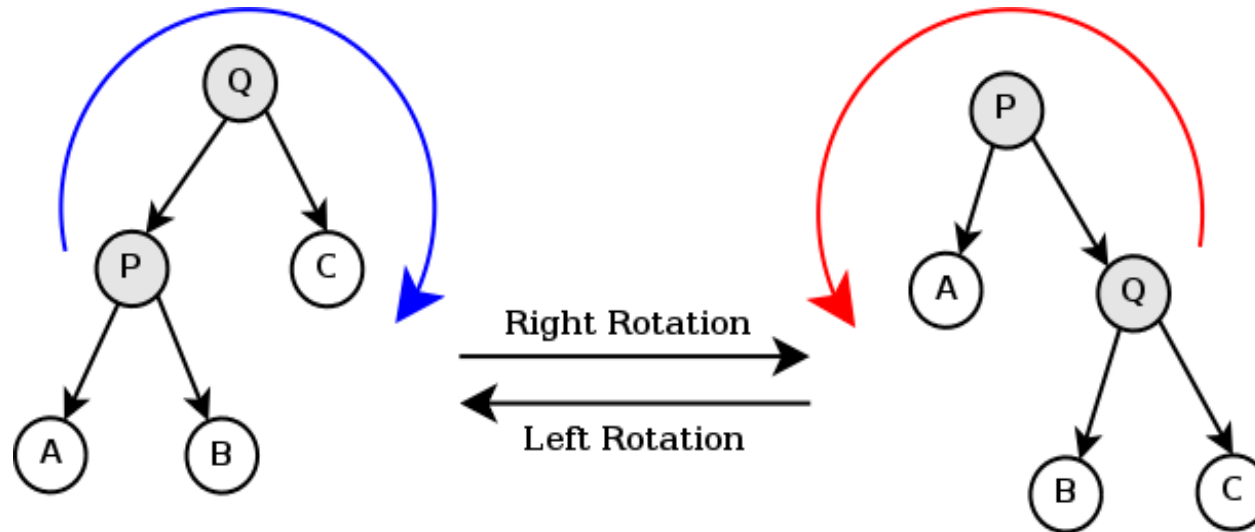
Tree Rotations

- A rotation is a local operation in a binary tree that changes the structure without violating the binary search tree (BST) property.
- It is used to rebalance trees after insertions or deletions.
- The tree balancing algorithms or techniques makes extensive use of rotations (rotating a node around another node).
- Tree rotations can be done either a left rotation or a right rotation.

Rotation Type	Used When	Effect
Right	Left-heavy tree	Promotes left child, demotes root to right child
Left	Right-heavy tree	Promotes right child, demotes root to left child

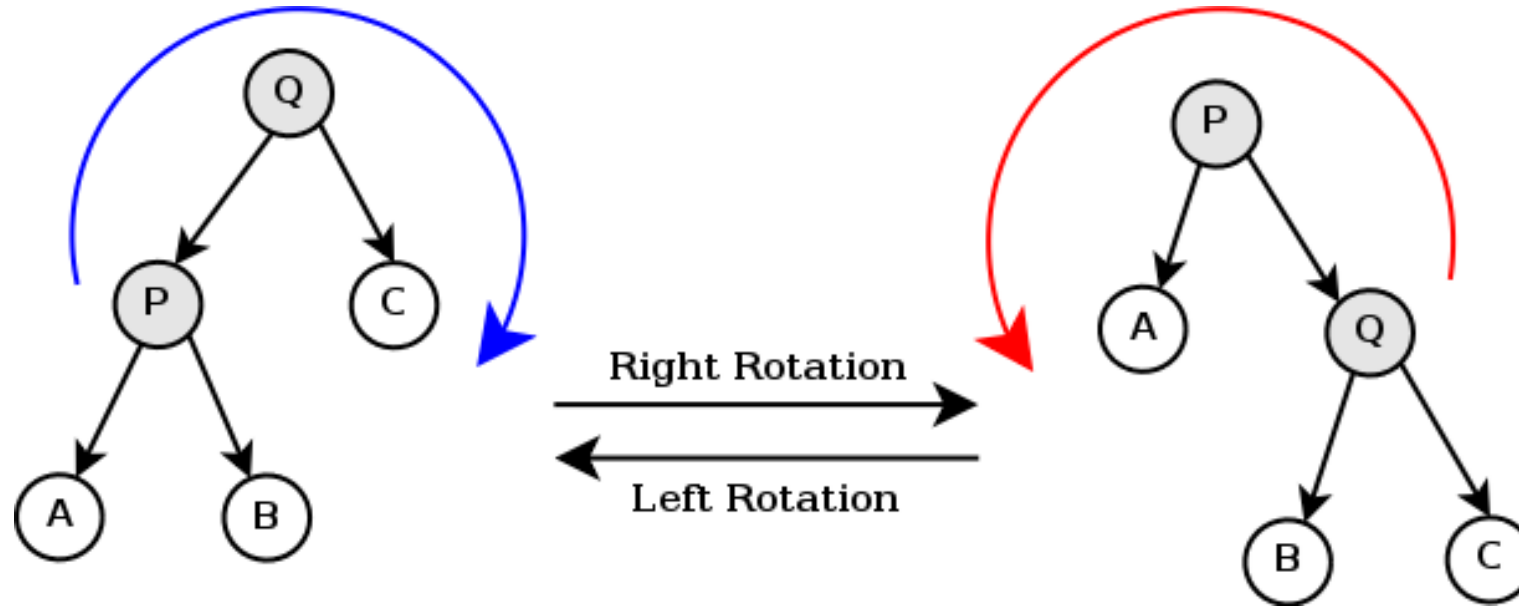


Right Rotation



- This is a transformation that moves left nodes to right. In the provided tree, node P is rotated around Q to take the place of node Q, while Q moves down to become P's right child.
- The subtree rooted at A remains unchanged.
- If P had a right child (in this case, B), it becomes the left child of Q.
- Right rotations are used in various self-balancing trees to maintain the tree's balance after insertions and deletions that skew the tree to the left.

Left Rotation



- This is the opposite transformation, where moves right nodes to left. Here, node P is rotated to become the left child of node Q.
- The subtree rooted at C remains unchanged.
- If Q had a left child before the rotation (which would be B in the case of a reverse of the right rotation), it becomes the right child of P.
- Left rotations are used to balance trees that are skewed to the right.

HOMework

Write a code to do right and left rotation in BST.

AVL VS. DSW TREES



How can we transform
an existing unbalanced BST
into a balanced BST?"



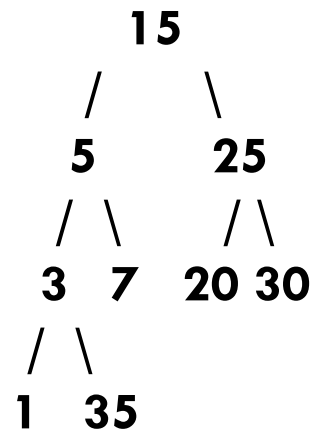
Solution to the BST balancing problem

DSW

DSW ALGORITHM

- The DSW (Day–Stout–Warren) algorithm is a method for balancing a binary search tree (BST) efficiently in linear time.
- It transforms an unbalanced BST into a perfect (if node count allows) or nearly perfect (if node count doesn't match the perfect tree formula) balanced tree.
- DSW balances any BST into a form where the height is minimized.
- Efficient: Runs in $O(n)$ time and uses $O(1)$ extra space.
- Suitable for applications where rebalancing is needed after bulk insertions.

The Day-Stout-Warren (DSW) algorithm aims to create a complete binary tree means a binary tree in which every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible.



DSW TREE BALANCING APPROACH

The DSW algorithm has two main steps and each phase is dominated by tree rotations.

1. Create a **vine (backbone)** – flatten the BST into a right-skewed linked list using right rotations. Every node has only a right child.
2. **Balance the vine (Compression)** – convert the vine into a balanced BST using left rotations.

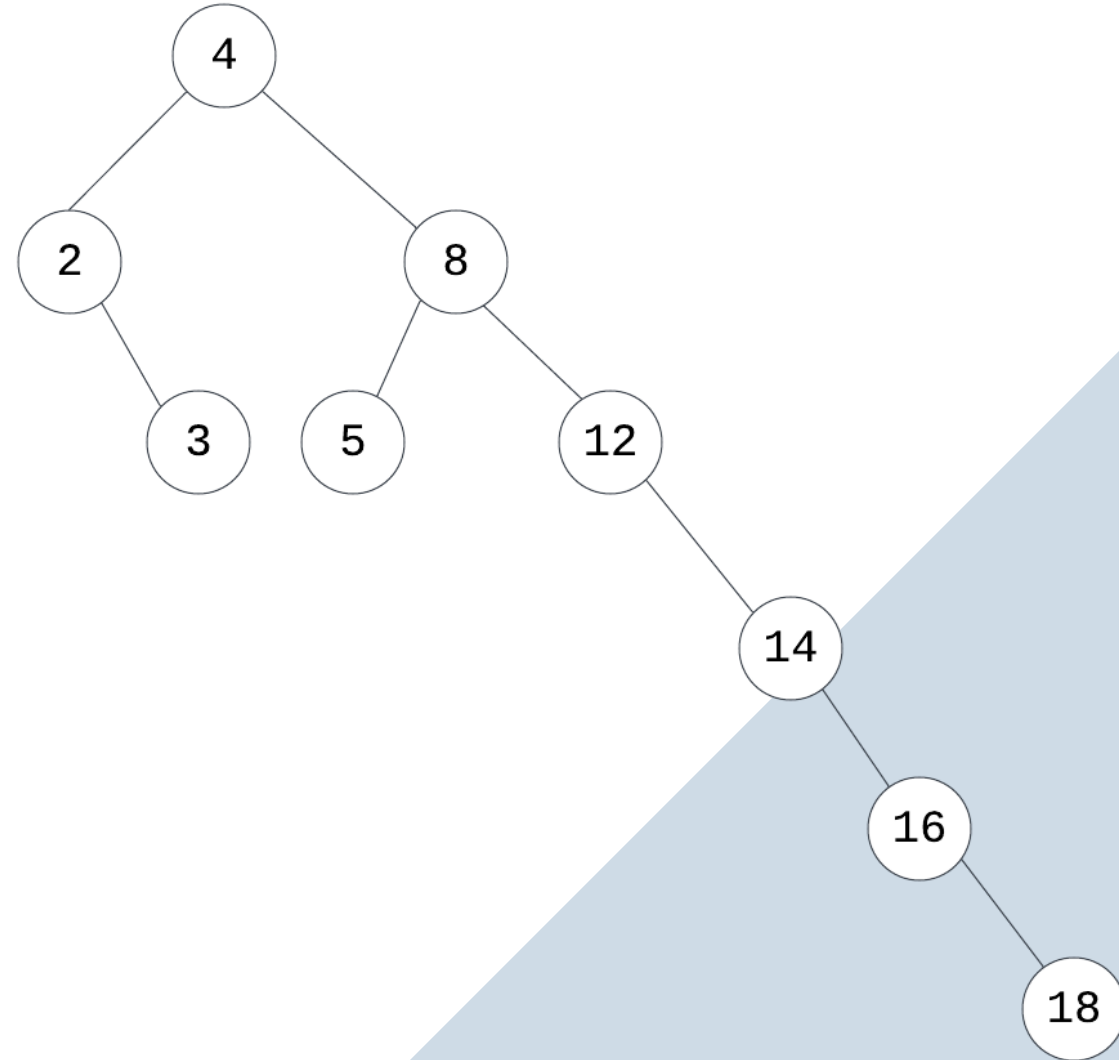
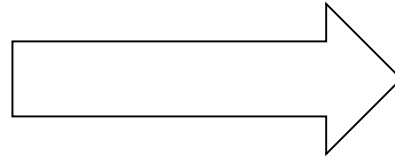
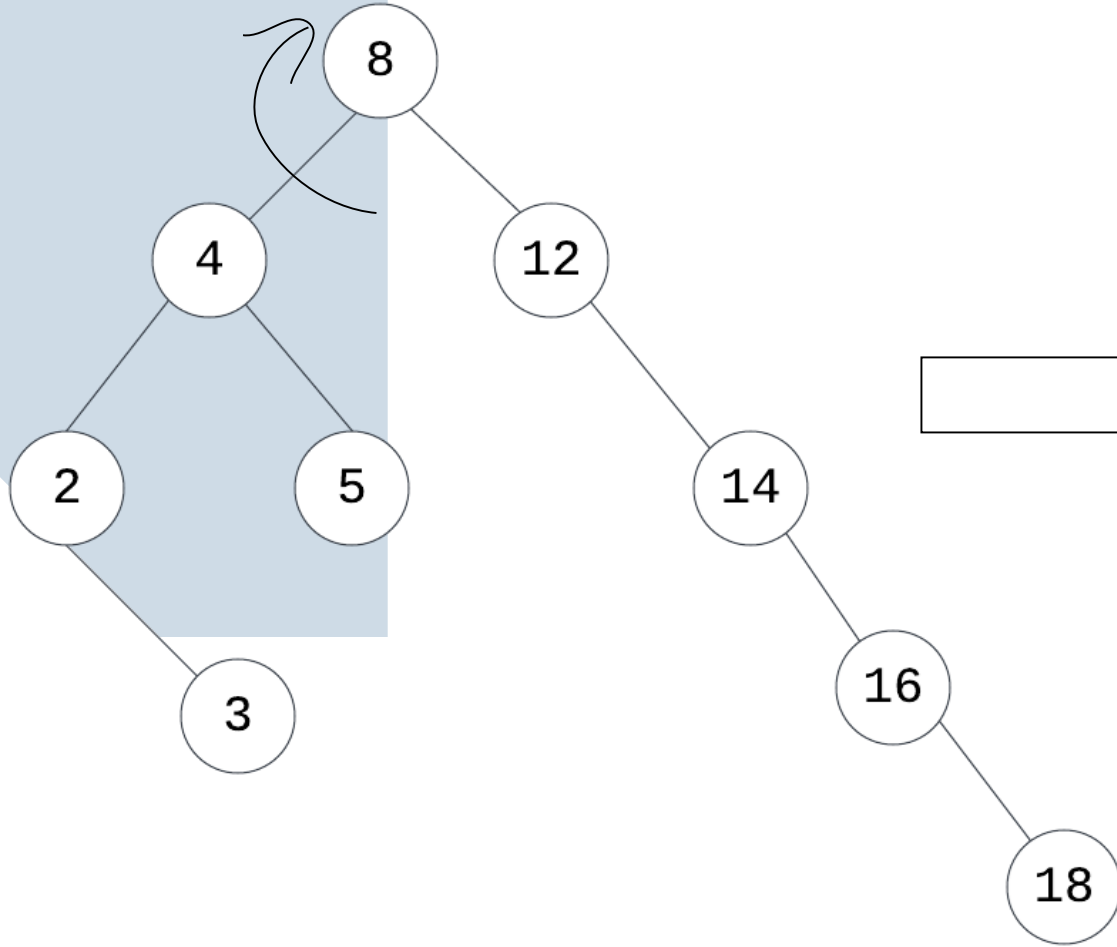
1. CREATE A BACKBONE:

- Objective: Transform the BST into a "vine" (a right-skewed tree where each node has only a right child).
- Method: Repeatedly apply **right rotations** to nodes with left children.

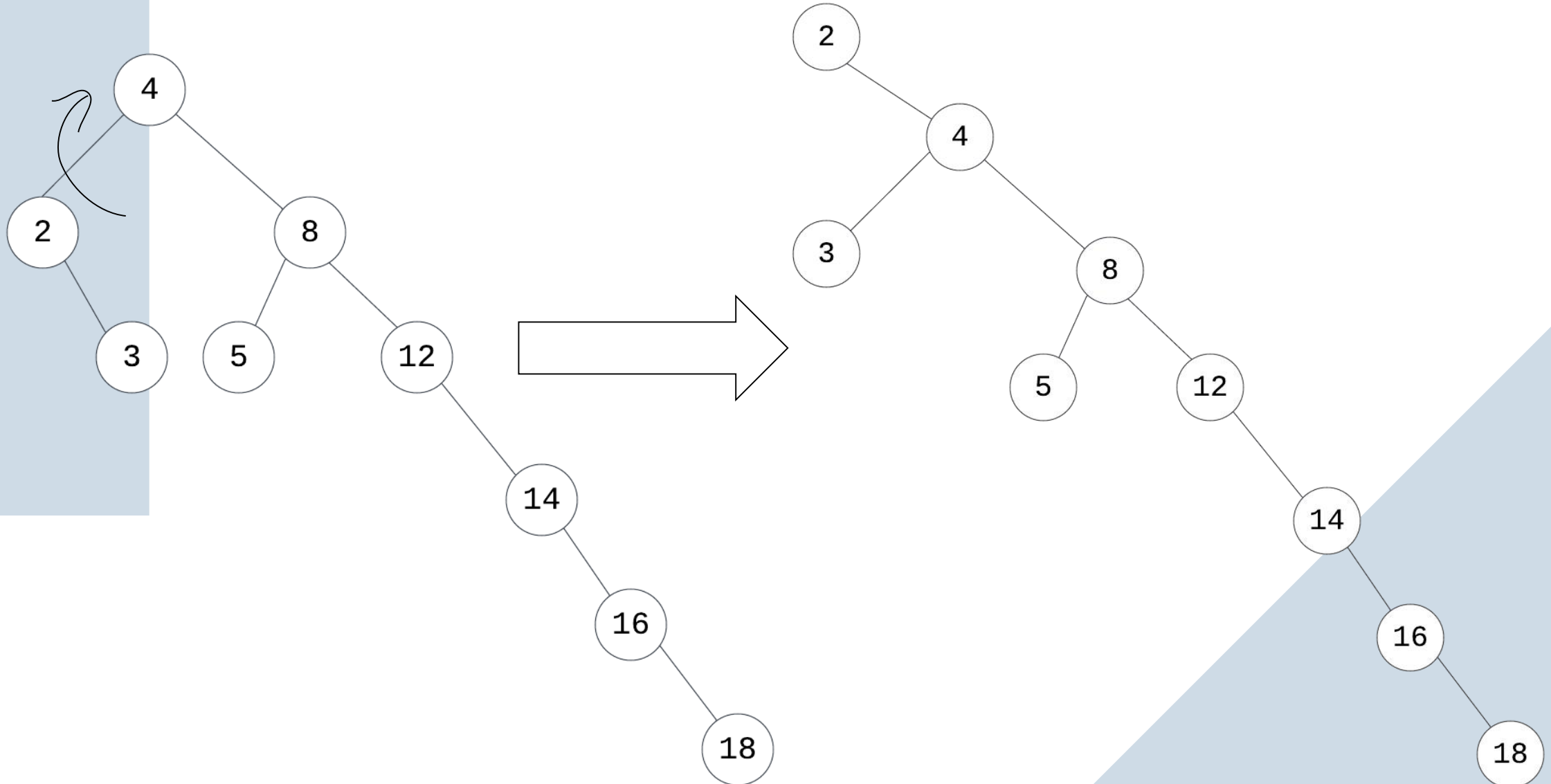
Steps:

- 1.Start at the root.
- 2.While traversing, if the current node has a left child:
 1. Perform a **right rotation**.
 2. Stay at the same node to check the new left child.
- 3.If there's no left child, move to the right child.
- 4.Repeat until the whole tree is transformed into a vine.

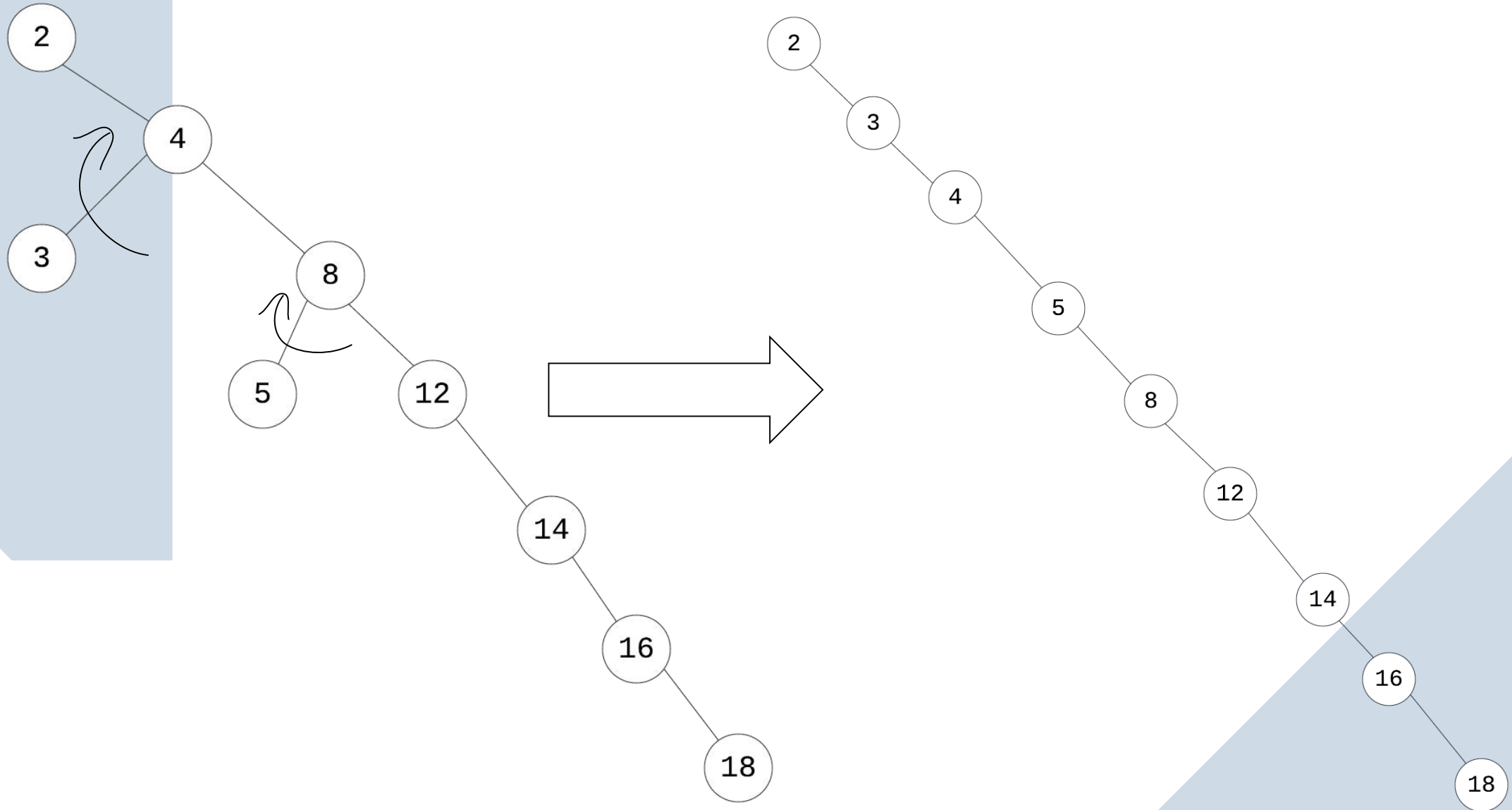
DSW Tree Balancing Approach



DSW Tree Balancing Approach



DSW Tree Balancing Approach



```
function createRightVine(Tree){  
    current = Tree.root  
    while(current != null){  
        if( current has leftChild)  
            right rotate leftChild around current  
            current = current.right  
        else  
            current = current.right  
    }  
}
```

HOMework

Write a C code to creating a Right vine Algorithm

2. BALANCING THE TREE

- Objective: Transform the right-skewed vine into a balanced BST using **left rotations**.
- Key idea: Apply left rotations in stages to bring nodes up to higher levels in a balanced fashion.

Steps:

- 1.Count the number of nodes (n) in the vine.
2. Need to find number of nodes we need to build a perfectly balanced tree.

A perfectly balanced tree represents an exact triangle. Which means a relationship can be derived between the height (h) of the perfectly balanced tree and the total number of nodes (n) in the tree.

$$n = 2^{h+1} - 1$$

$$h + 1 = \lg_2 (n + 1)$$

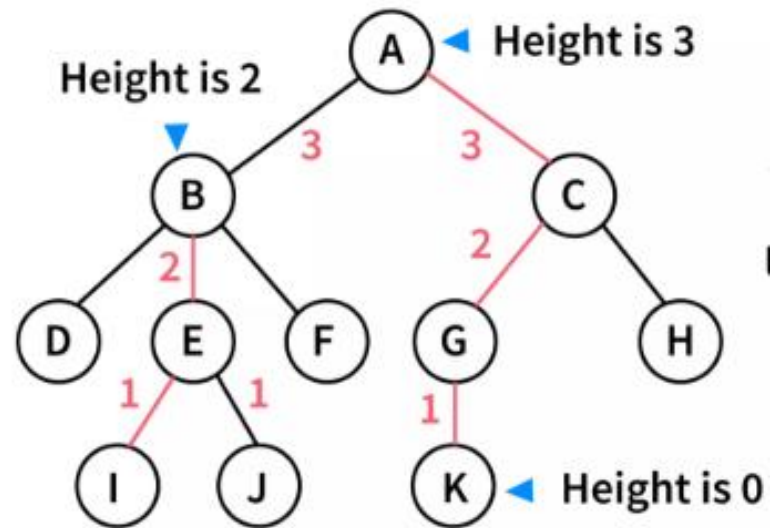
$$m = 2^{\text{floor}(\lg_2 (n+1)) - 1}$$

$$n = 2^h - 1$$
$$h = \lg_2 (n + 1)$$

$$m = 2^{\text{floor}(\lg_2 (n + 1)) - 1}$$

Height of a Node – number of edges in the longest path from the node to a leaf

Height of a Tree – height of its' root



Since the vine in above tree contains 9 nodes the closest height of a perfectly balanced tree is 3 ($\lg_2(10) \sim 3$). So the number nodes (m) in a perfectly balanced tree of height 3 as per the 1st equation

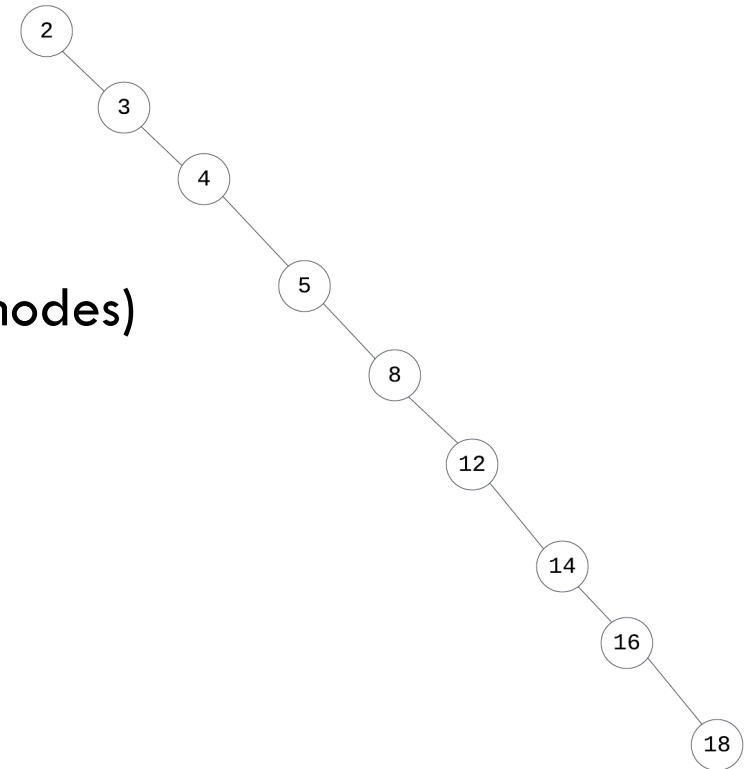
n = number of nodes in the tree (9 nodes)

m = number of nodes in the closest perfectly balanced tree (7 nodes)

Compute the largest $m = 2^{\text{floor}(\log_2(n+1))} - 1$

$\log_2(10) \approx 3.32$, so $\text{floor}(\log_2(10)) = 3$

$m = 2^3 - 1 = 7$



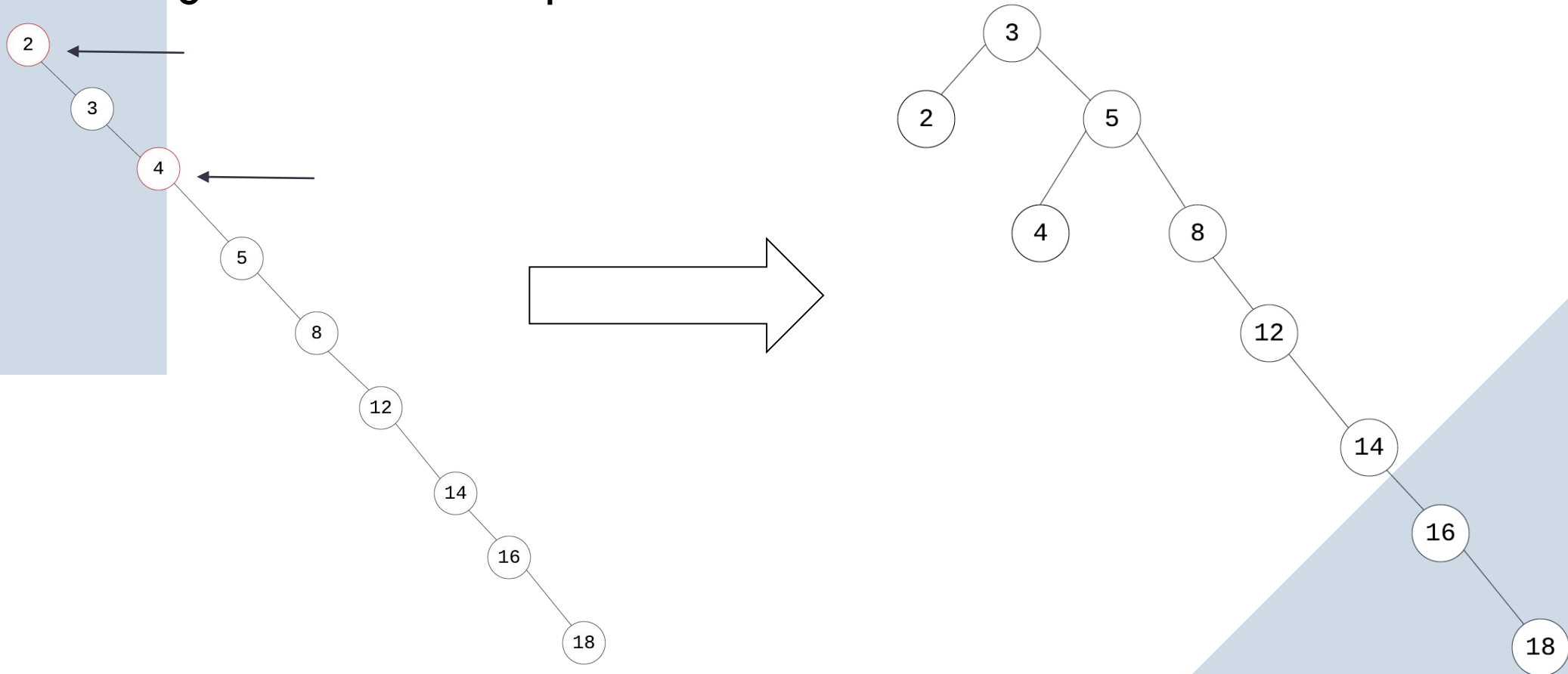
3. Perform $n-m$ left rotations starting from the root to shorten the vine.

- So the node difference between the perfect tree and the tree to be balanced can be calculated by $n-m$ which results in 2.
- This is called the excess of nodes to the perfectly balanced tree.
- In order to balance this excess nodes left rotation is performed on **$(n-m)$ odd nodes** in the vine.

excess of nodes to the perfectly balanced = $n-m$

DSW Tree Balancing Approach Contd

When we consider above example two left rotations(**n-m**) would be done initially. Since we are selecting odd nodes, it will be done on 1st and 3rd node according to above example it is on values 2 and 4





3. Then repeatedly:

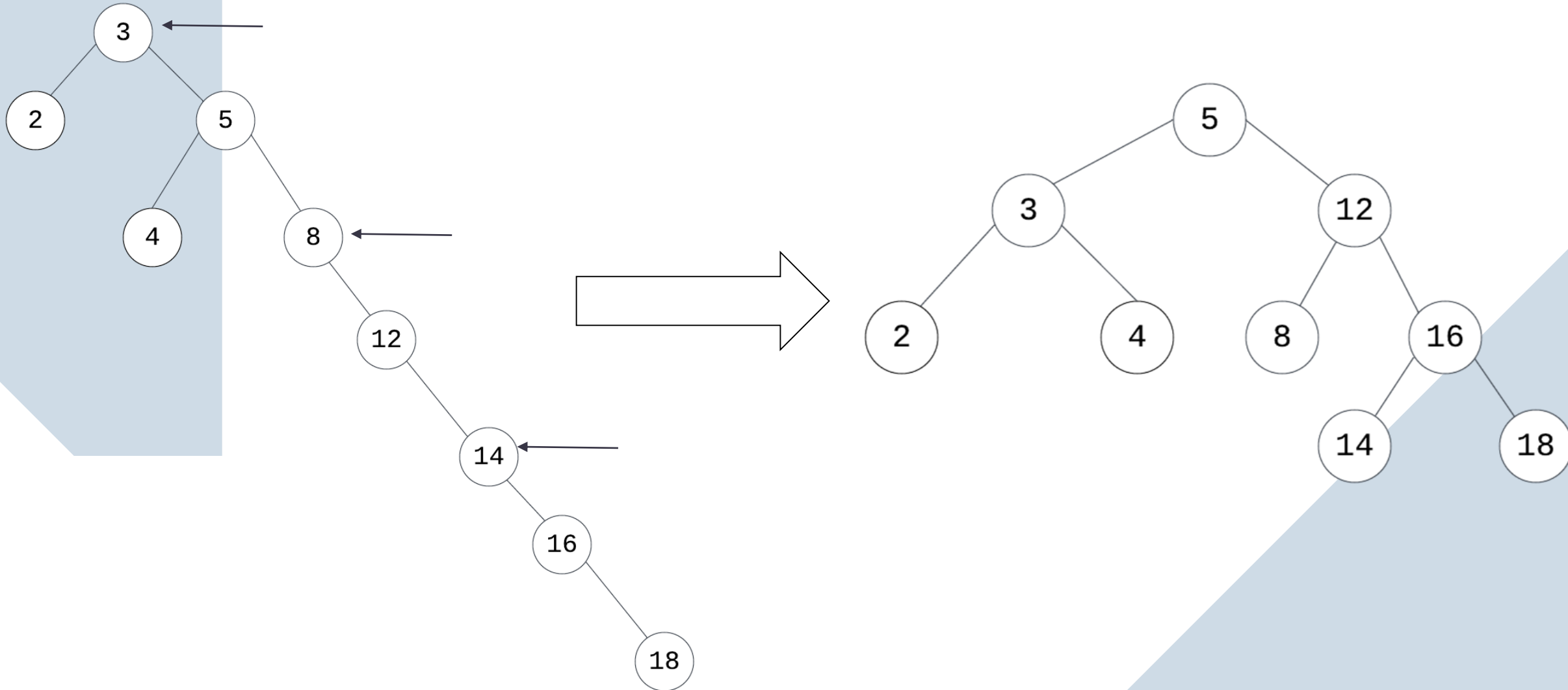
- Halve m
- Perform m left rotations, starting from the root.

5. Continue until $m = 0$.

DSW Tree Balancing Approach Contd

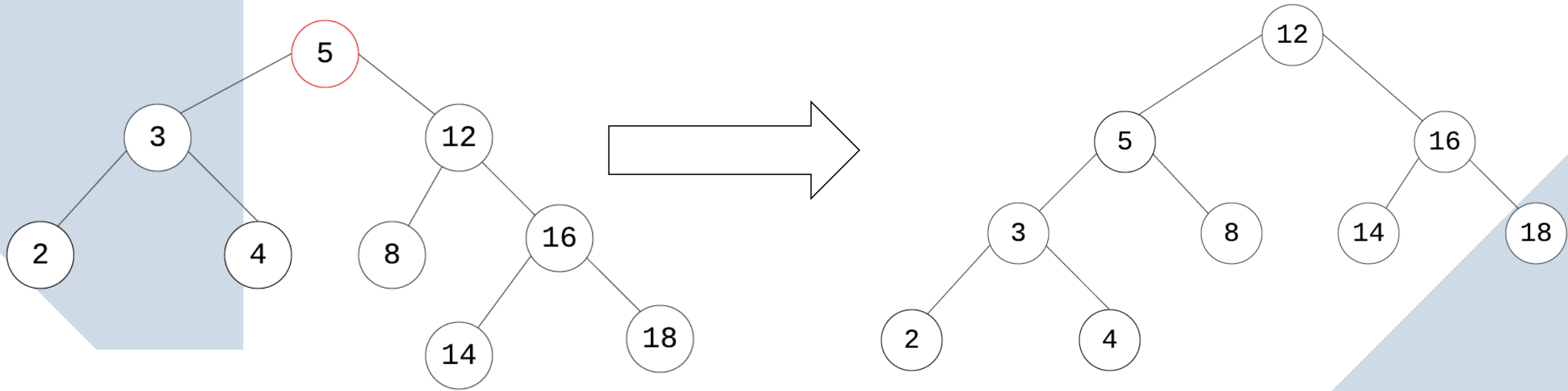
- After left rotation of excess nodes, next you have to half the size of the vine through each pass.
- Number of rotations to be performed is calculated by m which is halved in each iteration of the loop as $m = m/2$ (only take the integer value).
- So to start the final phase we recalculate m which is $(7/2 = 3)$. So 3 left rotations are performed to the vine

DSW Tree Balancing Approach Contd



DSW Tree Balancing Approach Contd

In the next iteration m further halved to 1. So one left rotation is performed on the one node in the vine which is value 5 in the above example.



If m was further divided the value will be lesser than 1. Therefore the height differences is now less than one or one. Therefore the tree has attained a balanced state. So the above tree is the final globally balanced tree.

STEPS

1. Start with an unbalanced BST.
2. Convert the BST into a vine (backbone) by repeatedly performing right rotations.
3. Count the number of nodes in the vine.
4. Calculate the number of rotations required.
5. Compress the vine by performing left rotations.
6. Repeat the compression step until the tree becomes balanced.
7. Return the balanced BST.

```
function CreateBalancedTree(n)
    m = 2^(floor(log2(n + 1))) - 1
    make (n-m) left rotations on odd or second node starting from root

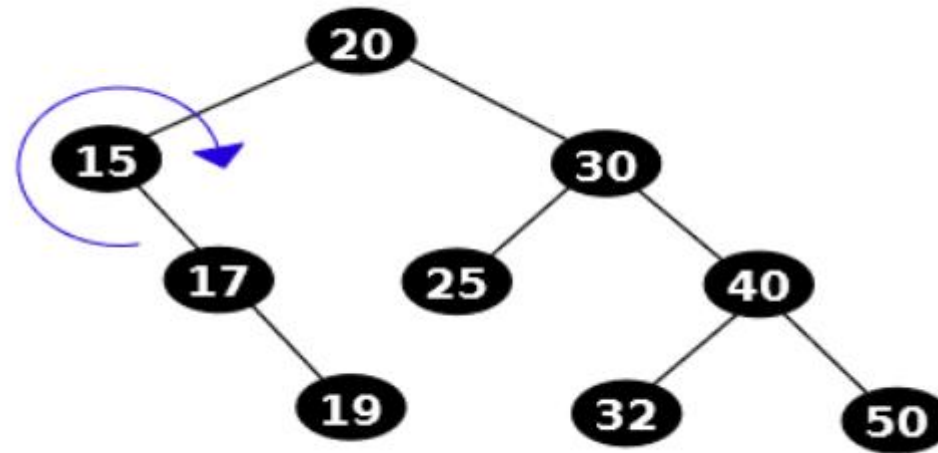
    while (m > 1)
        m = m / 2
        make (m) left rotations on odd or second node starting from root
    end while
end function
```

HOMework

Write a C code to Balancing the tree Algorithm

EXERCISE

Balance the following BST using DSW



PSEUDO CODE FOR RIGHT ROTATION

```
function RIGHT_ROTATE(Tree, Parent)
```

```
    Child ← Parent.left
```

```
    if Child = NULL then
        return
```

```
    OldParent ← Parent.parent
```

```
    Parent.left ← Child.left    ←
```

```
    if Child.right ≠ NULL then
        Child.right.parent ← Parent
```

```
    Child.right ← Parent
    Parent.parent ← Child
```

```
    Child.parent ← OldParent
```

```
    if OldParent = NULL then
        Tree.root ← Child
```

```
    else if OldParent.left = Parent then
        OldParent.left ← Child
```

```
    else
        OldParent.right ← Child
```

```
end function
```

Check this code and understand

```
function RIGHT_ROTATE(Tree, Parent)
```

```
    Child ← Parent.left
```

```
    if Child = NULL then
        return
```

```
    OldParent ← Parent.parent
```

```
    // Move Child's right subtree
    Parent.left ← Child.right
```

```
    if Child.right ≠ NULL then
        Child.right.parent ← Parent
```

```
    // Perform rotation
    Child.right ← Parent
    Parent.parent ← Child
```

```
    // Reattach rotated subtree
    Child.parent ← OldParent
```

```
    if OldParent = NULL then
        Tree.root ← Child
```

```
    else if OldParent.left = Parent then
        OldParent.left ← Child
```

```
    else
        OldParent.right ← Child
```

```
end function
```

```
function LEFT_ROTATE(Tree, Parent)
```

```
    Child ← Parent.right
```

```
    if Child = NULL then
```

```
        return
```

```
    OldParent ← Parent.parent
```

```
    // Move Child's left subtree
```

```
    Parent.right ← Child.left
```

```
    if Child.left ≠ NULL then
```

```
        Child.left.parent ← Parent
```

```
    // Perform rotation
```

```
    Child.left ← Parent
```

```
    Parent.parent ← Child
```

```
    // Reattach rotated subtree
```

```
    Child.parent ← OldParent
```

```
    if OldParent = NULL then
```

```
        Tree.root ← Child
```

```
    else if OldParent.left = Parent then
```

```
        OldParent.left ← Child
```

```
    else
```

```
        OldParent.right ← Child
```

```
end function
```

CREATE VINE

1. Start at the root.
2. If the current node has a left child, perform a right rotation.
3. Continue removing left children.
4. Repeat until no node has a left child.
5. The tree is now a right vine (backbone).

CREATE-VINE(Tree)

Grand ← NULL

Current ← Tree.root

while Current ≠ NULL

if Current.left ≠ NULL **then**

 RIGHT-ROTATE(Tree, Current)

if Grand = NULL **then**

 Current ← Tree.root

else

 Current ← Grand.right

else

 Grand ← Current

 Current ← Current.right

end while

end CREATE-VINE

COMPRESSION

1. Start at the root of the vine.
2. Perform a left rotation.
3. Move to the next pair of nodes.
4. Perform another left rotation.
5. Continue until the required number of rotations has been completed.
6. The vine becomes shorter and more balanced.
7. Repeat compression passes until the tree is balanced.

```
COMPRESS(Tree, Count)
```

```
    Grand ← NULL
```

```
    Current ← Tree.root
```

```
    for i ← 1 to Count do
```

```
        Child ← Current.right
```

```
        LEFT-ROTATE(Tree, Current)
```

```
        Grand ← Child
```

```
        Current ← Grand.right
```

```
    end for
```

```
end COMPRESS
```

DSW ALGO

```
DSW(Tree)

    CREATE-VINE(Tree)

    n ← Number of Nodes

    m ← 2 $\lceil \log_2(n+1) \rceil$  - 1

    COMPRESS(Tree, n - m)

    while m > 1 do

        m ←  $\lfloor m / 2 \rfloor$ 

        COMPRESS(Tree, m)

    end while

end DSW
```

multiway search trees

What if a node could store
multiple keys instead of one?

B Tree

Databases further optimize B-Trees
by storing actual records
only in leaf nodes
and linking leaves together

B+ Tree

THANK YOU !

I am available at : ysr@ucsc.cmb.ac.lk